ITWORX®

WORXFORGE / INTERNSHIP CASE STUDY

Juniors Can. Here's the Code.

Four students. Four weeks. One working application.

The WorxForge story: how ITWorx interns built with AI coding agents, learned a new stack, and took ownership with senior mentorship.

ITWorx Summer Internship - 2026

4	4 weeks	.NET + React
COMPUTER SCIENCE STUDENTS	INTERNSHIP DURATION	APPLICATION STACK

Hany Gheit

Senior Software Engineering Manager, ITWorx

Technical mentor to the WorxForge internship team

Publication version 1.5 | October 12, 2026

github.com/itworx/WorxForge

CASE OVERVIEW

Executive summary

The stack was new. The deadline was four weeks. The application worked.

In ITWorx's Summer 2026 internship, four computer science undergraduates offered a practical challenge to the assumption that AI coding agents leave little room for juniors. Most began with basic C# knowledge and limited React experience. Within four weeks, they built and locally demonstrated WorxForge using an ASP.NET Core API and React. [1]

WorxForge is a knowledge asset platform designed to help people publish, discover, and reuse prompts, agent instructions, workflows, templates, and coding standards. It gave the students connected user journeys to build across the interface, API, and data layers. [3]

The delivery approach connected learning with a clear target. Week one emphasized concepts and the big picture over memorizing syntax, alongside version control, testing, and agentic coding. Week two covered AI-assisted requirements and planning; week three, implementation using coding agents through a **GitHub Copilot Business subscription**; and week four, local deployment, testing, and presentation. The mentor reports that initial preparation took about one week, with some learners using an extra day, and the main implementation phase took one week. [1,2]

The agents wrote the application code. The interns directed the work, tested the results, and iterated toward the agreed behavior; they did not manually write that code. Senior guidance shaped the learning strategy, project scope, and decisions. Their mentor wrote no code, made no commits, and did not inspect the source for that first delivery. The interns owned the implementation. These are mentor-reported observations. [1]

At the final demonstration, the agreed workflows behaved as expected and the interface showed strong consistency. The mentor judged the students' delivery an excellent outcome for a stack new to them in practical work. His only notable reservation about their use of the agents was heavy token consumption, which he considered acceptable for first-time learners. That feedback is qualitative; no token totals or cost comparisons are available. [1]

This is one observational case, without a controlled comparison or independently audited code authorship. It suggests that concept-focused preparation and agentic tools can bring learners into useful project work sooner, alongside motivation, defined scope, and mentorship. It does not establish a general productivity gain, long-term learning outcomes, or production readiness.

After preparation for public release, WorxForge is now available under the MIT license at github.com/itworx/WorxForge. The repository gives the community a concrete project to inspect, improve, and build upon; its documentation describes the public release and remaining feature work. [6]

> The takeaway: juniors can build with AI agents when learning, ownership, and senior engineering judgment are part of the delivery model. WorxForge makes that possibility concrete.

01 / CASE STUDY

Team background and product scope

Undergraduate students learning a new stack

The team comprised four computer science students from different universities and faculties, at different stages of their undergraduate studies. In my account, this cohort was formed primarily through referrals, rather than selection based on a formal technical assessment. Most had basic C# knowledge and limited practical React experience. These descriptions reflect my firsthand observations, rather than standardized assessment results. [1]

Their curiosity and willingness to learn stood out to me. I guided the learning approach, helped frame the project, and supported decisions as the work progressed. The interns implemented the application and delivered the demonstration.

A knowledge asset platform

WorxForge was designed to help people publish, discover, and reuse knowledge assets such as prompts, agent instructions, workflows, templates, and coding standards. The project gave the interns connected user workflows to build across the interface, API, and data layers. [3]

The planned scope included asset metadata and content, discovery and filtering, saved libraries, profiles, collaboration, and notifications. Account and administration capabilities were also included in the requirements. The requirements defined the intended scope; they are not evidence that every listed capability was completed. [3]

The application combined an ASP.NET Core backend with a React frontend. The team's presentation also described React with TypeScript, clean architecture, dependency injection, and the repository pattern as part of their learning and project experience. [4]

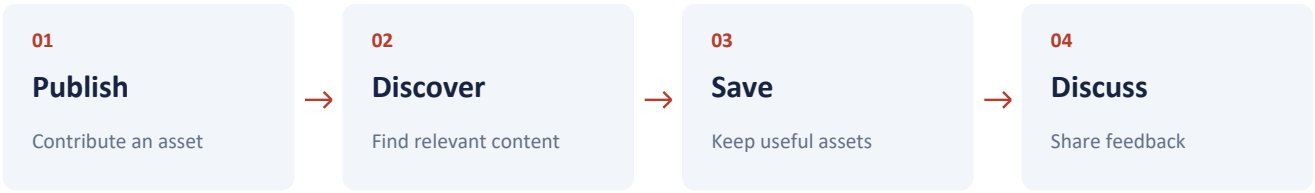


Figure 1. A schematic of the intended knowledge asset workflow. It illustrates the product concept and is not a screenshot or a verification of feature completeness.

At the final local demonstration, the application presented a consistent interface across discovery, publishing, saved assets, and activity notifications. The demonstration and supplied interface views support that observation; they do not independently establish reliability or security. Selected product views appear in Appendix C. [1,5]

02 / CASE STUDY

Learning and delivery in four weeks

The internship allocated one week each to initial learning, requirements and planning, main implementation, and testing and deployment. This sequence reflects my account of the project. The written plan also proposed earlier integration work during planning and quality activities throughout implementation. [1,2]

Week	Primary focus	Mentor-reported outcome
1	Concepts and guided self-study	Initial preparation completed in about one week. Some interns used an additional day.
2	Requirements and planning	Product requirements, repository preparation, and an implementation plan.
3	Agentic implementation with a **GitHub Copilot Business subscription**	Main implementation phase completed in one week.
4	Testing and deployment	A locally deployed application presented on the final day.

Concepts before memorization

In earlier internship programs, we had allocated at least a month to a broadly similar learning plan. For this cohort, we changed the emphasis. We asked the interns to focus on concepts and the big picture, with less time spent memorizing syntax or repeating implementation exercises. Most completed the initial learning phase within a week; some needed one additional day. [1]

The preparation used mostly free educational resources, including YouTube material. Topics included C# and .NET fundamentals, data access with Entity Framework Core, the ASP.NET Core request lifecycle, frontend prerequisites, React components and hooks, and GitHub Copilot. Supporting material covered architecture, unit testing, and version control. [1,2]

The aim was to help learners understand what the code should do and how its parts communicate, so they could guide implementation with AI assistance. Reading code, recognizing errors, and understanding behavior remained part of the learning task.

The plan included an explain-and-defend habit and proposed tasks with AI disabled to assess understanding. The case does not provide assessment scores or evidence that every planned assessment was completed. Those planned activities should therefore be distinguished from the reported delivery results. [2]

03 / CASE STUDY

Planning, implementation, and demonstration

Requirements gave the team a shared target

During the second week, the interns used AI assistants to help develop requirements and plan the application. They prepared the repository and the implementation approach. The project requirements gave them a shared description of the product and its workflows. [1,3]

Publishing a knowledge asset, for example, involves content, metadata, validation, persistence, and how the asset appears in discovery. Planning those relationships gives a learner a basis for directing a coding tool and assessing whether the resulting behavior meets the requirement.

The interns owned the initial implementation

According to my account, the interns were provided with a **GitHub Copilot Business subscription** and used GitHub Copilot's coding agents in a fully agentic workflow during the third week. They directed the agents to write the application code and did not manually write that code themselves. Their contribution was to guide implementation, test the results, and iterate toward the agreed behavior. This is a mentor-reported description; the paper does not independently audit code authorship or agent activity. [1]

Their final presentation described work from requirements through database design, backend, and frontend implementation. It also reported unit and integration testing, Git and GitHub collaboration, peer code review, and sprint planning. No independent historical test execution results or coverage measurements are established by this paper. [4]

For the version presented at the end of the internship, I did not write code, make commits, or inspect the source. My involvement was mentorship. The interns owned that implementation. Senior guidance in this case should therefore not be interpreted as a comprehensive senior code review. [1]

A working application in a local setting

The final week focused on deployment and testing in a local environment. On the last day, the interns presented the deployed application. In that demonstration, the agreed workflows behaved as expected, and I was impressed by the consistency and quality of the interface. [1]

Mentor feedback on delivery and token use

In my view, the students did an excellent job. They presented a fully working application for the agreed demonstration scope, using a stack that was new to them in practical project work. Their limited prior experience makes this a strong outcome for this cohort. [1]

> The only notable reservation I had about their use of the agents was heavy token consumption. I considered it acceptable for students working with agentic coding for the first time. This is a qualitative observation; the paper provides no measured token totals, cost figures, or comparable usage baseline. [1]

These observations concern the local demonstration. Establishing production readiness would require further evidence, including security review, broader testing, and operational validation.

04 / CASE STUDY

Findings and evidence limits

What the case supports

Four students with limited experience in the selected stack delivered a working local application during an internship lasting four weeks. My account places the initial learning phase at about one week and the main implementation phase at one week. The team used AI tools and retained responsibility for the first version they demonstrated. [1]

Evidence basis	Supported claim and boundary
Mentor account	Cohort background and selection, reported schedule, agentic workflow, mentor involvement, demonstration outcomes, and qualitative token-use feedback.
Project plan and requirements	Intended learning approach, delivery phases, and product scope. They describe plans and requirements.
Team presentation	Reported experience with architecture, testing, peer review, and implementation across application layers.
Demo interface views	Visible product screens and interface consistency. They do not independently verify complete workflows.

Earlier entry into practical work

Compared with my experience of earlier learning programs, this cohort reached practical project work sooner. Concept-focused preparation and AI assistance appeared to help them progress quickly, alongside mentorship, motivation, and a defined scope. [1]

The comparison concerns different programs and different learners. The case does not isolate the effect of AI or provide comparable effort records and learning assessments. It supports an observation of a shorter initial preparation period followed by useful delivery, without establishing a general speed improvement.

A bounded observational case

This paper concerns one cohort and one application. Referral-based selection and the absence of standardized baseline measurements limit how far the result can be generalized. It does not provide a controlled comparison, long-term maintenance results, defect-rate comparisons, or measured retention of skills. It also does not establish that every requirement was completed or that the internship application was ready for production.

The result gives a concrete reason to question the assumption that junior developers cannot contribute with AI coding agents. Repeating the approach with more cohorts and clearer measurements would help establish how consistently it works and how much mentorship is required.

05 / CASE STUDY

Implications for junior development programs

Combine early contribution with understanding

The experience suggests an approach for engineering managers, mentors, and educators: give learners a bounded product task, support implementation with AI, and examine both the delivered behavior and the learner's understanding. The following practices are recommendations for repeating and strengthening the approach; they are not additional claims about completed activities in this cohort.

1. **Tie concepts to the project.** Explain the request lifecycle, data flow, state, validation, and failure behavior through the application the learners will build. Use AI to support questions and examples, while checking the learner's explanation.
2. **Define observable acceptance criteria.** Agree on user workflows and expected results before implementation. A connected workflow helps learners understand the relationship between the interface, API, and data.
3. **Work in manageable increments.** Ask learners to explain a change, test it against the requirement, and discuss it with a peer. Evaluate the reasoning behind a change alongside its visible behavior.
4. **Assign senior review where it matters.** Review authorization, sensitive data handling, external services, and deployment behavior before relying on the application. This would strengthen the assurance process beyond the mentorship and demonstration reported here.
5. **Assess understanding independently.** Include explain, debug, and modify tasks with AI disabled, as the learning plan proposed. Record results so delivery speed can be evaluated alongside learning. [2]

Measure delivery and learning together

A future evaluation should record time to the first reviewed contribution, acceptance-test results, mentor effort, and defects found after integration. Record token consumption alongside these measures to assess how usage changes as learners gain experience. Independent explanation and debugging tasks would help assess understanding. Follow-up maintenance work would help show whether learners retained the skills.

The same approach may inform graduate development programs, but this cohort comprised undergraduate students. It does not directly measure outcomes for fresh graduates, all junior developers, or unmentored learners.

The opportunity is to bring learners into meaningful work earlier while helping them develop engineering judgment. The WorxForge case shows that AI assistance can be part of that process.

06 / CASE STUDY

WorxForge as an open-source project

WorxForge is publicly available under the MIT license:

github.com/itworx/WorxForge

The internship application and the public release are separate milestones. Release preparation included configuration cleanup, consolidation of the frontend and backend into a monorepo, and changes to onboarding and maintenance. Those later changes should be distinguished from what the interns built and demonstrated. [1,6]

The public version reviewed on October 12, 2026 uses a shared guest workspace without per-user isolation. Its documented capabilities include asset discovery and publishing, bookmarks, comments, Useful feedback, profiles, and in-app notifications. Account flows require further work, email delivery is disabled, and some administration capabilities remain unfinished. These limitations describe the public version, distinct from the historical demonstration. [6]

Contributions are welcome to complete account and administration flows, strengthen testing and accessibility, improve discovery, and make setup and maintenance clearer. Contributors should consult the repository documentation for the current feature status as the project evolves.

Acknowledgments

The four interns built the original application. Their commitment and willingness to learn made the experiment possible. I was proud to see them present a working application at the end of the internship.

Internship team

Amr Atif LinkedIn profile	**Mahmoud Tarek** LinkedIn profile
Aya Sherif LinkedIn profile	**Alyaa Eissa** LinkedIn profile

WorxForge offers a concrete example of students building with AI coding agents and mentorship. For teams developing junior talent, it is a useful case to examine and an approach worth evaluating further.

Evidence references

[1] Hany Gheit. Firsthand mentor account of the Summer 2026 cohort and its selection, schedule, agentic workflow, token use, involvement, and final demonstration. These descriptions and assessments are reported observations, not independently audited measurements.

[2] Internship learning and delivery plan. Concepts, explain-and-defend approach, quality activities, and proposed assessments. Planned activities are distinguished from verified outcomes.

[3] WorxForge project requirements. Product purpose, intended scope, and user workflows.

[4] Internship team presentation. Reported technical learning, testing, collaboration, and work across application layers.

[5] Internship demonstration interface views. Evidence of visible screens and layout consistency. Selected regions are reproduced in Appendix C, with private context cropped or permanently redacted.

[6] WorxForge public repository README, feature-status documentation, and MIT license, reviewed October 12, 2026. Current source status is distinct from the original internship delivery.

APPENDIX A / LEARNING PLAN

From learning to delivery

This public adaptation of the internship plan makes the approach reusable. The exercises below are suggested checkpoints, not verified results from this cohort. Select relevant lessons and adjust the pace to the learners' starting skills. [2]

Week 1: concepts, then application

Stage	Concepts and resources	Suggested checkpoint
Day 1	C# types, control flow, OOP, and runtime. R1.	Explain a simple model and trace its inputs and outputs.
Day 2	EF Core entities, DbContext, migrations, and queries. R2.	Explain data access and inspect a generated migration.
Day 3	API routing, controllers, dependency injection, and request flow. R3.	Trace a request through the API and data layer.
Day 4	Web basics, JavaScript, React components, state, and hooks. R4-R7.	Connect a view to an API and explain how its state changes.
Day 5	GitHub Copilot, task context, and directing agent work. R8.	Direct a small change, inspect the diff, run checks, and explain its behavior.
Depth	Architecture, tests, Git, advanced C#, and AI product context. R9-R14.	Choose targeted lessons as needed. Add a focused test and explain how a change fits the design.

Weeks 2-4: planned delivery activities

Week 2 - establish a shared target. Define requirements and user stories, agree on architecture and data/API contracts, prepare the repository, and connect the first workflow for review.

Week 3 - build and examine the result. Implement bounded features, integrate and review changes, refactor, add unit and integration tests, triage security findings, and practice an AI-disabled modification task.

Week 4 - prepare delivery and assess understanding. Test, resolve defects, document, prepare deployment, demonstrate the application, and assess independent explanation, debugging, and modification.

The original plan proposed reviews, security checks, and independent assessments; this case does not establish that every activity was completed. Use selected lessons rather than treating every full course as a one-week requirement. R12-R14 are optional enrichment. [2]

APPENDIX B / PUBLIC RESOURCES

Free YouTube learning resources

These public resources support reuse of the plan; they do not establish which lessons the interns completed. The list retains public resources from the original plan and adds alternatives for architecture, unit testing, and Git. Public video or playlist metadata was checked on October 12, 2026.

Topic and creator	Click to open the resource
R1 - C# fundamentals Passionate Coders	C# fundamentals playlist
R2 - Entity Framework Core DevCreed	EF Core introduction and playlist
R3 - ASP.NET Core API Julio Casal	ASP.NET Core full course for beginners
R4 - Web prerequisites devdojo	HTML, CSS, and JavaScript in 30 minutes
R5 - Modern JavaScript Programming with Mosh	ES6: modern JavaScript in 1 hour
R6 - JavaScript for React PedroTech	JavaScript you need to know for React
R7 - React components and hooks JavaScript Mastery	React JS 19 full course
R8 - GitHub Copilot GitHub	GitHub Copilot: a beginner's guide
R9 - Clean architecture Milan Jovanović	Clean Architecture project setup (.NET 7)
R10 - .NET unit testing Nick Chapsas	Writing tests in .NET using xUnit
R11 - Git and GitHub freeCodeCamp.org	Git and GitHub for Beginners
R12 - Advanced C# (optional) Passionate Coders	C# Advanced playlist
R13 - Copilot certification (optional) Microsoft Learn	GH-300: GitHub Copilot playlist
R14 - AI product context (optional) itichannel	How real AI products are designed and deployed

Use selected lessons; some material is in Arabic. Examples use different framework versions. Follow the repository's supported versions when applying examples, and check current commands and APIs.

APPENDIX C / PRODUCT OVERVIEW

A closer look at WorxForge

Four connected views show the internship application's discovery, publishing, saved assets, and notifications. The figures reproduce selected regions from the supplied demonstration captures. [5]

Discover useful knowledge

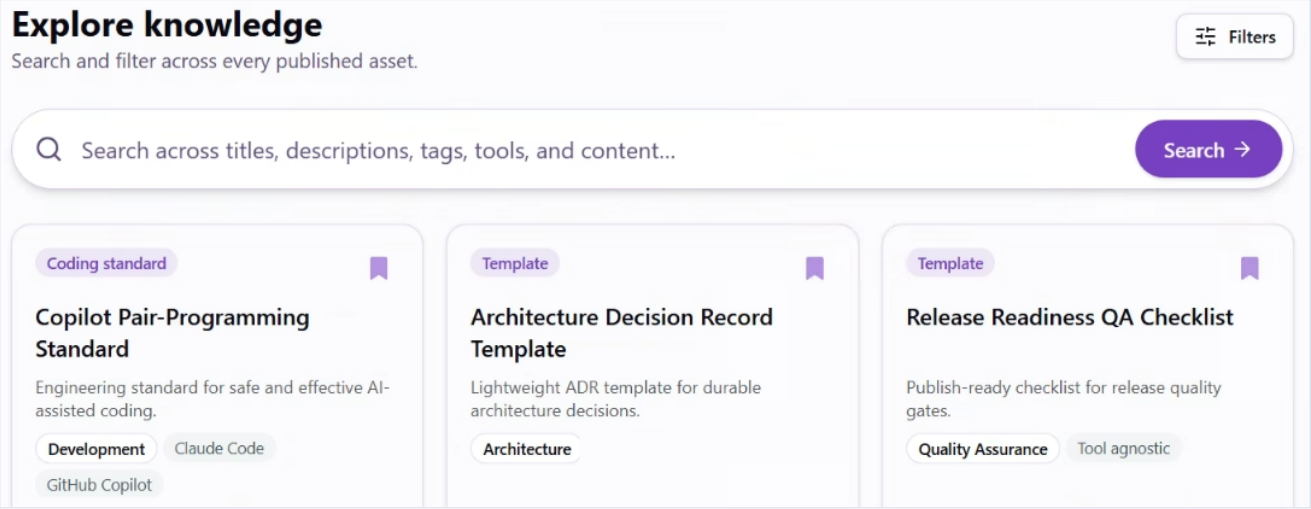


Figure 2. Discovery: search and filters above asset cards with types, descriptions, and bookmark controls.

Publish an asset

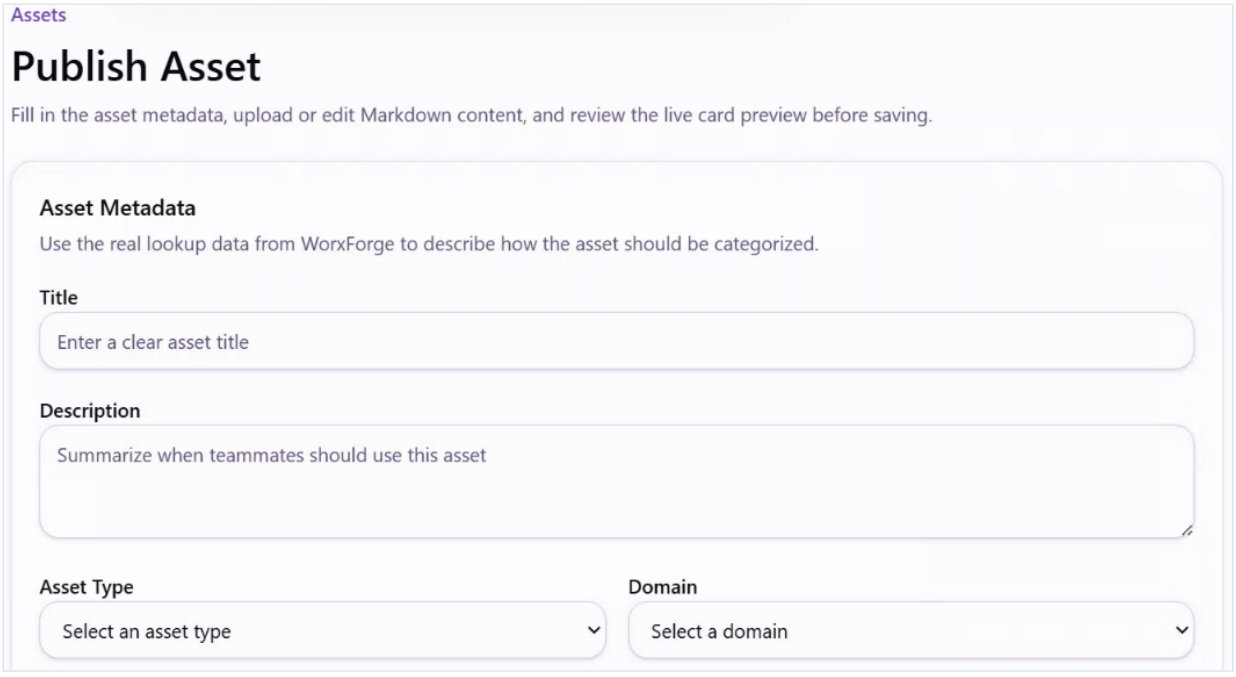


Figure 3. Publishing: title, description, asset type, and domain fields in the asset metadata form.

APPENDIX C / PRODUCT OVERVIEW

Save knowledge. Follow activity.

The demonstration also showed saved assets and activity notifications within the same visual design. These views illustrate the application's everyday knowledge-sharing workflows. [1,5]

Keep useful assets close

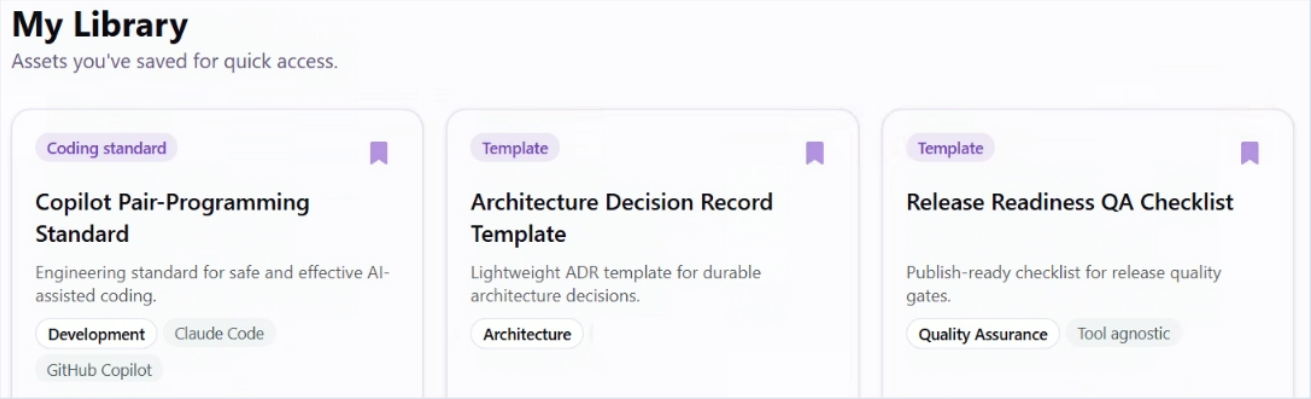


Figure 4. My Library: bookmarked assets collected for quick access.

Stay aware of activity

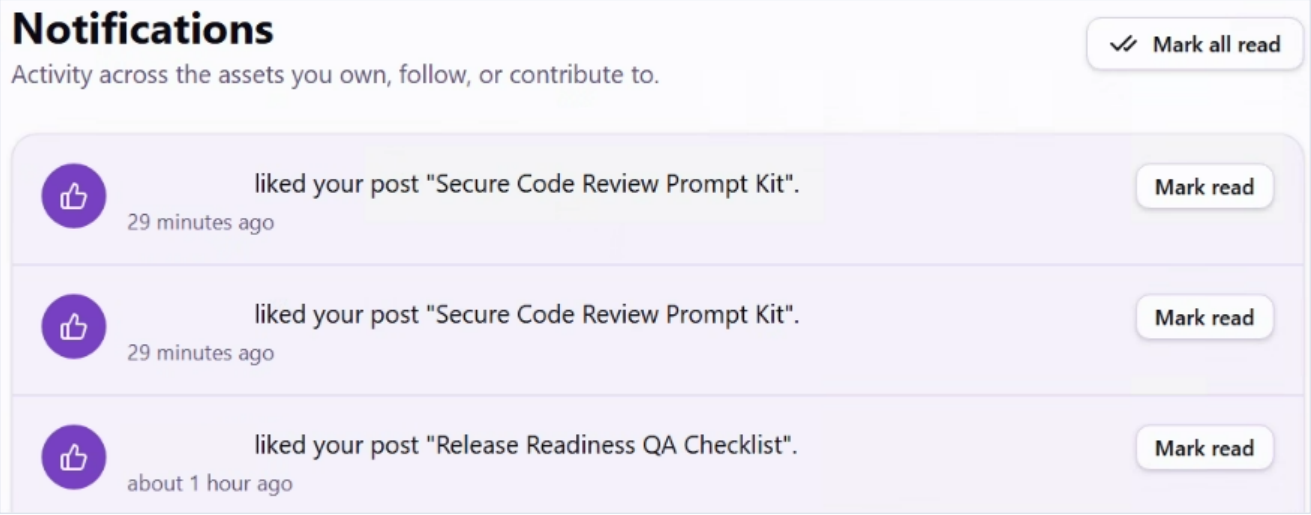


Figure 5. Notifications: an activity list with individual and bulk read controls. Person names are redacted.

These views document visible interface design; they do not independently prove complete behavior, security, or production readiness. [5]

Publication treatment: selected regions are cropped; personal identifiers and internal tool labels are permanently redacted. Browser, desktop, and meeting context are excluded.